

ACCESO A BASES DE DATOS DESDE PHP

El uso de bases de datos es imprescindible en una aplicación web. Además de establecer mecanismos de control de acceso a la web, guardar las preferencias de los usuarios, ofrecer información actualizada con frecuencia o buscar información, tiene muchas ventajas.

Ventajas

1. Conservar los datos de manera permanente
2. Usar herramientas proporcionadas por un SGBD para poder gestionar usuarios
3. Emplear herramientas proporcionadas por un SGBD para poder gestionar datos
4. Mejorar la seguridad de la web, ya que los datos son almacenados en otro servidor, el servidor de bases de datos
5. Usar el lenguaje estándar SQL en el manejo de los datos almacenados

Acceso a MySQL desde PHP

Además de los servidores con los que hemos trabajado hasta ahora, necesitaremos un SGBD de MySQL. Usaremos el de MariaDB que viene por defecto con XAMPP. Después tenemos que seguir los siguientes pasos:

1. **Establecer conexión con MySQL.** Lo podemos hacer con la API mysqli o con la [API PDO](#).
2. **Ejecutar** las sentencias.
3. **Cerrar la conexión.**

Establecer la conexión MySQL desde PHP con mysqli

La API `mysqli` ofrece, entre otras, las siguientes ventajas:

1. Interfaz orientada a objetos
2. Soporte para transacciones y diferentes declaraciones
3. Soporte de depuración mejorado
4. Funcionalidades más potentes

Necesitamos detallar la ip y el puerto donde se encuentra el SGBD, y el usuario y la contraseña para acceder. Si usamos XAMPP, no tendremos contraseña por defecto. Sería de la siguiente forma:

```
<?php
$servername = "localhost";
$username = "root";
$password = "root";

// Creamos conexión
$conn = new mysqli($servername, $username, $password);

// Check connection
if ($conn->connect_error) {
    die("La conexión ha fallado: " . $conn->connect_error);
}
echo "¡Nos hemos conectado!";
?>
```

Crear BD desde PHP

```
<?php
$servername = "localhost";
$username = "root";
$password = "root";

// Creamos la conexión
$conn = mysqli_connect($servername, $username, $password);
// Check connection
if (!$conn) {
    die("La conexión ha fallado: " . mysqli_connect_error());
}

// Creamos la BD llamada BD_IAW y comprobamos si se ha creado
//en cuyo caso la función mysqli_query nos devuelve true
$sql = "CREATE DATABASE BD_IAW";
if (mysqli_query($conn, $sql)) {
    echo "La BD se ha creado correctamente";
} else {
    echo "Error al crear la BD: " . mysqli_error($conn);
}

//Cerramos la conexión
mysqli_close($conn);
?>
```

Crear tabla en una BD desde PHP

```
<?php
$servername = "localhost";
$username = "root";
$password = "root";
$dbname = "BD_IAW";

// Creamos la conexión
$conn = mysqli_connect($servername, $username, $password, $dbname);
// Comprobamos conexión ahora añadiendo también la BD a la que nos
// queremos conectar
if (!$conn) {
    die("La conexión ha fallado: " . mysqli_connect_error());
}

// Creamos la sentencia SQL para la creación
// de la tabla persona en la BD que hemos creado BD_IAW
$sql = "CREATE TABLE persona (
    id INT(6) UNSIGNED AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(30) NOT NULL,
    apellido VARCHAR(30) NOT NULL,
    email VARCHAR(50),
    fechaRegistro TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
)";
// Ejecutamos la sentencia sql frente a la BD y si se crea correctamente
// mostramos un mensaje
if (mysqli_query($conn, $sql)) {
    echo "La BD se ha creado correctamente";
} else {
    echo "Error al crear la BD: " . mysqli_error($conn);
}
//Cerramos la conexión
mysqli_close($conn);
?>
```


Insertar datos en tablas de BD desde PHP(1/2)

```
<?php
$servername = "localhost";
$username = "root";
$password = "root";
$dbname = "BD_IAW";

// Creamos la conexión
$conn = mysqli_connect($servername, $username, $password, $dbname);
// Comprobamos conexión ahora añadiendo también la BD a la que nos
// queremos conectar
if (!$conn) {
    die("La conexión ha fallado: " . mysqli_connect_error());
}

// Creamos la sentencia SQL para la inserción
// de una persona a la tabla persona de la BD creada
$sql = "INSERT INTO persona (nombre, apellido, email)
VALUES ('Jon', 'Saenz', 'jon@example.com')";

// Ejecutamos la sentencia sql frente a la BD y si se crea correctamente
// mostramos un mensaje
if (mysqli_query($conn, $sql)) {
    echo "Se ha insertado correctamente";
} else {
    echo "Error al insertar datos: " . mysqli_error($conn);
}
//Cerramos la conexión
mysqli_close($conn);
?>
```

Insertar datos en tablas de BD desde PHP(2/2)

Si queremos saber el último id que ha sido insertado, lo podemos saber con la función **mysqli_insert_id** sobre la conexión. Cuando comprobamos que se ha ejecutado correctamente la sentencia, podemos mostrarlo:

```
// Ejecutamos la sentencia sql frente a la BD y si se crea correctamente
// mostramos un mensaje
if (mysqli_query($conn, $sql)) {
    $ultimo_id = mysqli_insert_id($conn);
    echo "Nueva persona insertada con id: " . $ultimo_id;
} else {
    echo "Error al insertar datos: " . mysqli_error($conn);
}
```

También **podemos insertar más de una fila** con **mysqli_multi_query** y concatenando las cadenas de sentencia.

IMPORTANTE: Cada sentencia debe acabar con “;” de SQL también, ¡no olvidarlo!

```
// Creamos la sentencia SQL para la inserción
// de varias personas a la tabla persona de la BD creada
// y vamos añadiendo todo el código a la sentencia.
$sql = "INSERT INTO persona (nombre, apellido, email)
VALUES ('Jon', 'Saenz', 'jon@example.com');";
$sql .= "INSERT INTO persona (nombre, apellido, email)
VALUES ('Maria', 'Pozo', 'maria@example.com');";
$sql .= "INSERT INTO persona (nombre, apellido, email)
VALUES ('Lorena', 'Cast', 'lorena@example.com');";

// Ejecutamos la sentencia sql frente a la BD y si se crea correctamente
// mostramos un mensaje
if (mysqli_multi_query($conn, $sql)) {
    echo "Se han insertado nuevos registros";
} else {
    echo "Error al insertar datos: " . mysqli_error($conn);
}
```

Consultas SQL desde PHP

El método **mysqli_num_rows** devuelve el número de filas que ha devuelto la consulta.

El método **mysqli_fetch_assoc** almacena una fila como si fuera un puntero. Cada vez que se ejecuta va avanzando una fila de las que devuelve la consulta. La primera vez que lo llamamos se colocará en la primera fila.

Podemos añadir a la consulta WHERE, ORDER BY, LIMIT, ... Se recorre de igual modo.

```
// Creamos la consulta SQL
$sql = "SELECT id, nombre, apellido FROM persona;";
// La ejecutamos almacenando las filas de respuesta en una variable
$result = mysqli_query($conn, $sql);

// Si la consulta ha devuelto alguna fila...
if (mysqli_num_rows($result) > 0) {
    // vamos recorriendo cada fila que devuelve la consulta y mostramos los datos
    //Cada fila es como un array asociativo al que podemos acceder a su valor
    //a través del nombre de la columna
    while($fila = mysqli_fetch_assoc($result)) {
        echo "id: " . $fila["id"]. " - Nombre: " . $fila["nombre"]. " " . $fila["apellido"]. "<br>";
    }
} else {
    echo "No ha devuelto ninguna fila";
}
```


Borrar datos en SQL desde PHP

```
<?php
$servername = "localhost";
$username = "root";
$password = "root";
$dbname = "BD_IAW";

// Creamos la conexión
$conn = mysqli_connect($servername, $username, $password, $dbname);
// Comprobamos conexión ahora añadiendo también la BD a la que nos
// queremos conectar
if (!$conn) {
    die("La conexión ha fallado: " . mysqli_connect_error());
}

// Creamos la consulta SQL para borrar
$sql = "DELETE FROM persona WHERE id=3";

// Si se ha borrado correctamente mostramos un mensaje
if (mysqli_query($conn, $sql)) {
    echo "La/s fila/s se ha/n borrado correctamente";
} else {
    echo "Error al intentar borrar en la tabla: " . mysqli_error($conn);
}

//Cerramos la conexión
mysqli_close($conn);
?>
```


Actualizar datos en SQL desde PHP

```
<?php
$servername = "localhost";
$username = "root";
$password = "root";
$dbname = "BD_IAW";

// Creamos la conexión
$conn = mysqli_connect($servername, $username, $password, $dbname);
// Comprobamos conexión ahora añadiendo también la BD a la que nos
// queremos conectar
if (!$conn) {
    die("La conexión ha fallado: " . mysqli_connect_error());
}

// Creamos la consulta SQL para actualizar
$sql = "UPDATE persona SET apellido='Garcia' WHERE id=2";

// Si se ha actualizado correctamente mostramos un mensaje
if (mysqli_query($conn, $sql)) {
    echo "La/s fila/s se ha/n actualizado correctamente";
} else {
    echo "Error al intentar actualizar en la tabla: " . mysqli_error($conn);
}

//Cerramos la conexión
mysqli_close($conn);
?>
```

Sentencias preparadas

Cuando directamente ponemos un valor en una consulta, en una actualización, en una inserción o en un borrado, puede que, al pasar esos datos a través de una interfaz gráfica, intenten hacer un ataque SQL Injection. Para evitarlo, se usan sentencias preparadas en las que no ponemos directamente el valor, sino que lo sustituimos por un **interrogante** (?) y más adelante, le damos el valor que queremos.

Para saber más sobre este tipo de sentencias y ver ejemplos pincha [aquí](#).

Cookies

A menudo se utiliza una cookie **para identificar a un usuario**. Una cookie es un pequeño archivo que el servidor inserta en la computadora del usuario. Cada vez que el mismo ordenador solicite una página con un navegador, también enviará la cookie. Con PHP, puedes crear y recuperar valores de cookies.

Se crea una cookie con la función **setcookie()** cuya sintaxis es:

```
setcookie(name, value, expire, path, domain, secure, httponly);
```

Sólo se requiere el parámetro *de nombre*. Todos los demás parámetros son opcionales.

Crear/Recuperar una cookie

El siguiente ejemplo crea una cookie denominada "usuario" con el valor "María Sainz". La cookie caducará después de 30 días (86400 * 30). El "/" significa que la cookie está disponible en todo el sitio web (en caso contrario, seleccione el directorio que prefiera).

Luego recuperamos el valor de la cookie "usuario" (usando la variable global `$_COOKIE`). También utilizamos `isset()` para saber si la cookie está configurada:

```
<?php
$cookie_name = "user";
$cookie_value = "Maria Sainz";
setcookie($cookie_name, $cookie_value, time() + (86400 * 30), "/"); // 86400 = 1 día

if(!isset($_COOKIE[$cookie_name])) {
    echo "La cookie llamada '" . $cookie_name . "' no está configurada!";
} else {
    echo "La cookie '" . $cookie_name . "' está configurada.<br>";
    echo "Su valor es: " . $_COOKIE[$cookie_name];
}
?>
```

Modificar el valor de una cookie

Se modifica usando de nuevo la función **setcookie()** cambiando los valores de la cookie que se identifica por el nombre.

Eliminar una cookie

Para eliminar una cookie, se utiliza la función la **setcookie()** con fecha de caducidad anterior:

```
<?php
// modificamos la fecha de expiración a una hora antes
setcookie("user", "", time() - 3600);

echo "La cookie 'user' ha sido borrada.";
?>
```

Comprobar que una cookie está habilitada

```
<?php
if(count($_COOKIE) > 0) {
    echo "Las cookies están habilitadas.";
} else {
    echo "Las cookies están deshabilitadas.";
}
?>
```


Sesiones PHP (1/3)

Cuando trabajas con una aplicación, la abres, haces algunos cambios y luego la cierras. Esto es muy parecido a una sesión. El ordenador sabe quién eres. Sabe cuándo inicias la aplicación y cuándo finaliza. Pero en Internet hay un problema: el servidor web no sabe quién eres ni qué haces, porque **la dirección HTTP no mantiene el estado**.

Las variables de sesión resuelven este problema almacenando información del usuario para utilizarla en varias páginas (por ejemplo, nombre de usuario, color favorito, etc.). De forma predeterminada, las variables de sesión **duran hasta que el usuario cierra el navegador**.

Por lo que las variables de sesión **contienen información sobre un único usuario y están disponibles para todas las páginas de una aplicación**.

Se inicia una sesión con la función **session_start()**. Las variables de sesión se configuran con la variable global de PHP **\$_SESSION**.

IMPORTANTE: session_start() debe ser lo primero en su documento. Antes de cualquier etiqueta HTML.

```
<?php
// Iniciamos la sesión
session_start();

// Configuramos variables de sesión
$_SESSION["favcolor"] = "rojo";
$_SESSION["favanimal"] = "tortuga";
echo "Las variables de sesión están configuradas.";
?>
```

Sesiones PHP (2/3)

Las variables de sesión no se pasan individualmente a cada página nueva, sino que se recuperan de la sesión que abrimos al principio de cada página (session_start()).

```
<?php
// Iniciamos la sesión
session_start();

// Mostramos las variables de sesión
echo "El color favorito es ", $_SESSION["favcolor"], "<br>";
echo "El animal favorito es ", $_SESSION["favanimal"], "<br>";
?>
```

Si queremos **modificar** una variable de sesión, sólo tenemos que sobreescribirla.

Para **eliminar todas las variables** de sesión globales y destruir la sesión: **session_unset()** y **session_destroy()**

```
<?php
// borra todas las variables de sesión
session_unset();

// destruye la sesión
session_destroy();
?>
```